



UT0. FUNDAMENTOS DE ACCESO A DATOS Y ENTORNO DE DESARROLLO

RA1: ACTIVIDAD 1

¡De ficheros a bases de datos! La evolución en el Acceso a Datos.

Descripción:

En esta primera actividad del módulo de **Acceso a Datos**, exploraremos las distintas **estrategias para almacenar y gestionar información**, desde los métodos más tradicionales, como los **ficheros**, hasta las **bases de datos más modernas**.

El objetivo es que adquieras una visión global y comprendas la **evolución y las motivaciones** que han llevado al desarrollo de las diferentes tecnologías de acceso a datos.

Objetivos:

- Identificar y comprender las principales **estrategias de acceso a datos**.
- Analizar **ventajas e inconvenientes** usando ficheros para gestionar información.
- Diferenciar entre distintos tipos de **bases de datos** (relacionales, orientadas a objetos y objeto-relacionales).
- Comprender el contexto histórico y las motivaciones que han impulsado el desarrollo de cada tipo de base de datos.
- Valorar la **idoneidad de cada estrategia** en función de los requisitos del proyecto.

Instrucciones:

Responde a las siguientes preguntas de forma clara y concisa.

1. Identifica diferentes **estrategias de Acceso a datos**.
2. Describe las **ventajas de las Bases de Datos frente a los ficheros**.
3. ¿Hay algún método de acceso a datos **mejor que otro** de manera categórica? Justifica tu respuesta.
4. Enumera y explica las **desventajas del uso de ficheros** para almacenar la información.
5. ¿Qué son los **ficheros XML**? ¿Para qué se utilizan?
6. Describe las principales **ventajas e inconvenientes de las Bases de Datos Relacionales (BDR)**.
7. ¿**Por qué surgen las Bases de Datos Orientadas a Objetos (BDOO)**?
8. Compara las **ventajas e inconvenientes de las BDOO frente a las BDR**.
9. ¿Qué son las **Bases de Datos Objeto-Relacionales (BDOR)**?
10. ¿Qué **aportaciones** introducen las BDOR respecto a las BDR?

Entrega:

1. **Entrega un único documento pdf con tus respuestas.**

RESPUESTAS...

1. Estrategias de acceso a datos

- Acceso secuencial: Lee o escribe registros de principio a fin, uno tras otro. Es eficiente para procesar lotes enteros, pero lento para búsquedas puntuales.
- Acceso directo o aleatorio: Permite situarse inmediatamente en un registro específico mediante una clave, desplazamiento (offset) o cálculo hash sin recorrer los anteriores.
- Acceso indexado: Combina una estructura de índices (árboles B/B+, tablas hash) con los datos subyacentes, permitiendo búsquedas rápidas tanto exactas como por rangos.
- Mapeo Objeto-Relacional (ORM): Capa de abstracción que traduce las entidades y relaciones de la base de datos a objetos en el código (ej. Hibernate, JPA).
- APIs / Servicios (REST, GraphQL): Acceso desacoplado donde las aplicaciones consumen datos mediante llamadas HTTP serializadas en formatos como JSON o XML.

2. Ventajas de las Bases de Datos frente a los ficheros

- Independencia física y lógica: El programa no necesita conocer la estructura física en disco ni la codificación interna de los datos para operar con ellos.
- Control de concurrencia: Múltiples usuarios pueden leer y escribir simultáneamente sin sobrescribir datos ajenos ni bloquear el sistema completo.
- Integridad y restricciones: El motor valida tipos, claves primarias, claves foráneas y reglas de negocio automáticamente.
- Seguridad y permisos: Permite definir privilegios detallados por usuario, tabla, columna o vista.
- Transacciones ACID: Garantiza atomicidad, consistencia, aislamiento y durabilidad ante fallos del sistema.

3. ¿Existe un método de acceso a datos categóricamente superior?

No. No existe un método absoluto; la elección depende estrictamente del caso de uso, el volumen de datos, la latencia requerida y la arquitectura:

- Para registrar eventos continuos (*logs*) o procesar flujos en tiempo real, el acceso secuencial o estructuras basadas en ficheros planos/append-only superan en velocidad a un motor relacional.
- Para consultas complejas con múltiples relaciones y consistencia crítica (banca, facturación), una BDR con acceso indexado es la opción adecuada.
- Para datos semiestructurados con esquemas dinámicos a escala masiva, bases de datos NoSQL o almacenes de documentos ofrecen mejor rendimiento y escalabilidad horizontal que un modelo relacional estricto.

4. Desventajas del uso de ficheros para almacenar información

- Redundancia e inconsistencia: Los datos suelen duplicarse en distintos archivos; si se actualizan en uno y no en otro, la información queda desincronizada.
- Dificultad de acceso y consultas: Requiere programar desde cero algoritmos de filtrado, ordenación y búsqueda para cada nueva necesidad de información.
- Aislamiento de los datos: Los formatos suelen estar dispersos en estructuras incompatibles, complicando la combinación de datos de distintas fuentes.
- Anomalías por concurrencia: Si dos procesos abren el mismo fichero en modo escritura, uno puede sobrescribir los cambios del otro o corromper el archivo.
- Falta de atomicidad: Si el sistema falla a mitad de una escritura, el fichero queda en un estado inconsistente o dañado sin mecanismo nativo de marcha atrás (*rollback*).



- Seguridad precaria: La protección suele depender únicamente de los permisos básicos del sistema operativo, sin filtrado a nivel de campo o registro.

5. Ficheros XML: definición y usos

- Qué son: Archivos de texto plano estructurados mediante etiquetas jerárquicas personalizables bajo el estándar eXtensible Markup Language. Son legibles tanto por humanos como por máquinas y autodescriptivos.
- Para qué se utilizan:
 - Intercambio de datos entre plataformas heterogéneas.
 - Archivos de configuración de aplicaciones y entornos (ej. pom.xml, web.xml).
 - Estructuras de mensajería para servicios web tradicionales.
 - Exportación e importación de copias de seguridad de datos estructurados.

6. Ventajas e inconvenientes de las Bases de Datos Relacionales (BDR)

- Ventajas:
 - Madurez tecnológica, estandarización (lenguaje SQL) y soporte masivo.
 - Garantía estricta de consistencia mediante propiedades ACID.
 - Modelo formal contrastado que minimiza duplicidades gracias a la normalización.
- Inconvenientes:
 - Desajuste de impedancia objeto-relacional: Fricción técnica al mapear objetos del lenguaje de programación a tablas y filas relacionales.
 - Dificultad para escalar horizontalmente (sharding complejo frente a la escalabilidad vertical habitual).
 - Estructura rígida: los cambios en esquemas grandes requieren migraciones costosas.

7. ¿Por qué surgen las Bases de Datos Orientadas a Objetos (BDOO)?

Surgen a finales de los años 80 por el auge de la Programación Orientada a Objetos (POO) (C++, Smalltalk, Java) y la necesidad de gestionar datos con estructuras complejas (sistemas CAD/CAM, GIS, gráficos 3D). Las BDR obligaban a descomponer objetos complejos en multitud de tablas vinculadas mediante *joins*, lo que degradaba el rendimiento y obligaba a escribir complejas capas intermedias de traducción para persistir y reconstruir el estado de las clases.

8. Comparativa: BDOO frente a BDR

Característica	Base de Datos Orientada a Objetos (BDOO)	Base de Datos Relacional (BDR)
Mapeo de clases	Directo; persiste objetos sin traducción ni pérdida de tipos.	Requiere capas de mapeo (ORM) o código SQL manual.
Modelado de datos	Soporta de forma nativa herencia, polimorfismo y encapsulación.	Tablas, columnas, claves primarias y foráneas; no soporta herencia nativa directa.



Rendimiento en grafos/complejidad	Muy rápido al navegar por punteros directos entre objetos complejos.	Se degrada si se requieren múltiples operaciones de unión sucesivas.
Estandarización	Baja.	Muy alta; estándar SQL universal con variantes menores.
Consultas ad-hoc	Complejas de formular; interfaces menos directas para analítica general.	Altamente eficientes y flexibles mediante sentencias declarativas SQL.
Mercado y soporte	Nicho específico; menor ecosistema de herramientas y administradores.	Estándar de la industria, ecosistema masivo y herramientas de monitorización avanzadas.

9. ¿Qué son las Bases de Datos Objeto-Relacionales (BDOR)?

Son una evolución de los sistemas relacionales tradicionales (SGBD) que extienden el modelo relacional incorporando características propias del paradigma orientado a objetos. Permiten seguir utilizando tablas, claves y SQL, pero añadiendo la capacidad de definir tipos de datos de usuario complejos (UDTs), herencia de tablas o tipos, métodos y referencias directas.

10. Aportaciones de las BDOR respecto a las BDR tradicionales

- Tipos de datos definidos por el usuario (UDTs): Permite crear estructuras complejas (vectores, direcciones completas, geometrías espaciales) tratadas como tipos nativos dentro de una columna.
- Soporte de herencia: Posibilidad de crear subtipos que heredan atributos de tipos padre, e incluso tablas que heredan de otras tablas.
- Encapsulación y métodos: Capacidad de asociar funciones y procedimientos directamente a los tipos de datos en la base de datos.
- Manejo de identificadores de objeto (OIDs): Referencias directas entre tuplas sin depender exclusivamente de claves primarias artificiales.
- Compatibilidad retroactiva: Mantienen la infraestructura SQL existente, el motor transaccional robusto y las herramientas de reporting de las BDR sin obligar a una migración disruptiva a modelos no estándar.